



**COMSATS University Islamabad**

**Lahore Campus**

## **FINAL LAB PROJECT REPORT**

### **Hostel Kharcha Manager**

*Data Structures and Algorithms (DSA)*

| Field                | Details                                           |
|----------------------|---------------------------------------------------|
| Course Title         | Data Structures and Algorithms                    |
| Submitted To         | Mrs. Humera Faisal                                |
| Group Members        | Aizaz Noor, Abdul Basit Shahid, Muhammad Abdullah |
| Registration Numbers | SP25-BSE-013, SP25-BSE-001, SP25-BSE-067          |
| Section              | A                                                 |
| Submission Date      | 20 May 2026                                       |

# Contents

|                                                    |    |
|----------------------------------------------------|----|
| 1. Introduction .....                              | 3  |
| 1.1 Problem Statement .....                        | 3  |
| 1.2 Objectives .....                               | 3  |
| 2. Introduction and Features of the Project .....  | 3  |
| 3. Team and Module Distribution .....              | 3  |
| 4. Data Structures and Algorithms Used .....       | 4  |
| 4.1 Hash Map - MemberHash .....                    | 4  |
| 4.2 Doubly Linked List - TransactionTimeline ..... | 4  |
| 4.3 Stack - Undo Engine .....                      | 4  |
| 4.4 Max-Heap - DebtHeap .....                      | 4  |
| 5. Output / Screenshots of Project Execution ..... | 5  |
| 5.1 Saved Data Files Output .....                  | 7  |
| members.csv sample .....                           | 7  |
| transactions.csv sample .....                      | 8  |
| 6. System Workflow .....                           | 8  |
| 7. Complexity Analysis .....                       | 8  |
| 8. Conclusions and Findings .....                  | 8  |
| 9. Challenges Faced .....                          | 9  |
| 10. Potential Future Enhancements .....            | 9  |
| 11. Contribution of Each Group Member .....        | 9  |
| Source Code: MemberHash.h .....                    | 10 |
| Source Code: MemberHash.cpp .....                  | 11 |
| Source Code: TransactionTimeline.h .....           | 14 |
| Source Code: TransactionTimeline.cpp .....         | 16 |
| Source Code: DebtHeap.h .....                      | 19 |
| Source Code: DebtHeap.cpp .....                    | 20 |
| Source Code: main.cpp .....                        | 23 |
| Data Files .....                                   | 34 |
| members.csv .....                                  | 34 |
| transactions.csv .....                             | 34 |

# 1. Introduction

Hostel Kharcha Manager is a console-based C++ application developed for the DSA Final Lab Project. Its purpose is to help hostel roommates manage shared expenses, deposits, refunds, balances, debt settlement, transaction history, and debt prioritization in a systematic way.

The project was selected because hostel expense sharing is a real-world problem where multiple data structures can be meaningfully applied. Member lookup requires fast searching, transaction history requires ordered traversal, undo requires last-in-first-out behavior, and debt prioritization requires extracting the maximum debtor efficiently.

The application uses custom implementations of Hash Map, Doubly Linked List, Stack, and Max-Heap. These structures are implemented manually using pointers and dynamic memory allocation rather than relying on STL containers for the core DSA logic.

## 1.1 Problem Statement

In hostel life, roommates frequently combine money for groceries, electricity, water, rent, internet, and food. Manual calculation through notebooks or WhatsApp messages can cause mistakes, forgotten payments, and disputes. Hostel Kharcha Manager solves this by recording every transaction, updating balances automatically, supporting undo, and showing who owes the most.

## 1.2 Objectives

- Develop a menu-driven C++ console application for hostel expense management.
- Demonstrate practical use of multiple DSA concepts in one integrated project.
- Implement custom data structures manually using classes, structs, pointers, new, and delete.
- Provide fast member registration, searching, balance update, transaction logging, undo, and debt ranking.
- Prepare the group for oral examination by documenting the complete code and logic.

# 2. Introduction and Features of the Project

| Feature                     | Description                                                                                        |
|-----------------------------|----------------------------------------------------------------------------------------------------|
| Member Registration / Login | Stores each roommate by ID, name, password, and balance. Uses MemberHash for fast lookup.          |
| Remove Roommate             | Allows safe deletion only when balance is settled, with refund or final settlement flow if needed. |
| Log Shared Expense          | Records a new expense, validates payer, accepts description and amount, and splits the expense.    |
| Equal Split                 | Divides total expense equally among all current members.                                           |
| Custom Weighted Split       | Allows different members to have different shares according to entered weights.                    |
| Transaction Timeline        | Stores all transactions chronologically using a doubly linked list.                                |
| Undo Last Transaction       | Uses a stack to undo the most recent transaction and reverse balance changes.                      |
| Roommate Balances           | Prints all members with current balance and status: REMAINED, OWES, or SETTLED.                    |
| Debt Priority Report        | Uses a max-heap to show members who owe the most money first.                                      |
| Data Persistence            | Saves members and transactions to CSV files and loads them when the program starts.                |

# 3. Team and Module Distribution

| Group Member               | Module / Responsibility                                                                 | Data Structure                  |
|----------------------------|-----------------------------------------------------------------------------------------|---------------------------------|
| Muhammad Abdullah / Abdul  | Member profile management, registration, login, balance printing, hash table operations | Hash Map with Separate Chaining |
| Aizaz Noor                 | Transaction history, chronological ledger, undo engine, transaction node design         | Doubly Linked List + Stack      |
| Abdul Basit Shahid / Basit | Debt ranking system, debtor report, priority analytics                                  | Max-Heap / Priority Queue       |
| All Members                | Integration in main.cpp, testing, debugging, final report, viva preparation             | Combined system workflow        |

## 4. Data Structures and Algorithms Used

### 4.1 Hash Map - MemberHash

MemberHash stores roommate profiles. The table has 100 buckets, and each bucket points to a linked list chain. The hash function sums ASCII values of the member ID and takes modulo 100. If two IDs map to the same bucket, separate chaining handles the collision.

| Operation         | Average Complexity | Worst Complexity | Reason                                                                 |
|-------------------|--------------------|------------------|------------------------------------------------------------------------|
| Add member        | $O(1)$             | $O(n)$           | Usually inserts into a short bucket chain; worst case all IDs collide. |
| Get member        | $O(1)$             | $O(n)$           | Direct bucket access followed by chain traversal only if needed.       |
| Remove member     | $O(1)$             | $O(n)$           | Finds the node in a bucket chain and relinks pointers.                 |
| Print all members | $O(n)$             | $O(n)$           | Must scan every bucket and every member node.                          |

### 4.2 Doubly Linked List - TransactionTimeline

TransactionTimeline stores expense records in chronological order. Each TransactionNode contains prev and next pointers, so the system can move forward and backward through transactions and delete a node efficiently when its address is known.

### 4.3 Stack - Undo Engine

Every logged transaction is pushed onto a linked-list-based stack. Undo pops the top stack node, gets the pointer to the latest TransactionNode, reverses its balance changes, removes it from the doubly linked list, and deletes the node from memory.

### 4.4 Max-Heap - DebtHeap

DebtHeap stores debtor records in an array-based binary max-heap. The member with the highest amount owed is always located at index 0. heapifyUp is used after insertion and heapifyDown is used after extracting the maximum debtor.

| Data Structure     | Project Use           | Why Appropriate                                      |
|--------------------|-----------------------|------------------------------------------------------|
| Hash Map           | Member lookup by ID   | Fast average $O(1)$ search and insertion.            |
| Doubly Linked List | Transaction timeline  | Maintains order and supports pointer-based deletion. |
| Stack              | Undo last transaction | LIFO matches undo behavior exactly.                  |
| Max-Heap           | Who owes the most     | Extracts highest debtor efficiently.                 |

## 5. Output / Screenshots of Project Execution

The following screenshots show the expected console execution flow of the project. They demonstrate portal login, main menu, balance table, transaction timeline, and debt priority report.

```
Command Prompt - .\HKM.exe

+-----+
|               HOSTEL KHARCHA MANAGER PORTAL               |
+-----+
| [ 1 ] Log In (Roommate or Admin)      [Access System] |
| [ 2 ] Sign Up (Register Roommate)    [Join System]   |
| [ 3 ] Exit Portal                     [Quit App]      |
+-----+

Your choice:
```

Figure 1: Portal login screen and admin access flow.

```
Command Prompt - .\HKM.exe

+-----+
|               HOSTEL KHARCHA MANAGER               |
|           Members: 5 | Transactions: 6             |
+-----+

+-----+
|               MAIN MENU               |
+-----+
| [ 1 ] Add New Roommate      [Profile] |
| [ 2 ] Remove a Roommate    [Delete]  |
| [ 3 ] Log a Shared Expense [Ledger]   |
| [ 4 ] Undo Last Transaction [Revert]  |
| [ 5 ] View Transaction Timeline [History] |
| [ 6 ] View Roommate Balances [Finances] |
| [ 7 ] Who Owes The Most?    [Analytics] |
| [ 8 ] Settle Debt / Deposit [Clear]   |
| [ 9 ] Exit & Save Data      |
+-----+

Your choice: _
```

Figure 2: Main menu after successful login.

```
Command Prompt - .\HKM.exe

  /_/_/_/  /_/_/_/  /_/_/_/
 /_/_/_/  /_/_/_/  /_/_/_/
/_/_/_/_/  /_/_/_/_/  /_/_/_/_/

H O S T E L   K H A R C H A   M A N A G E R

System Features:
[1] Fast Member Registration    -> Manage profiles & deposits
[2] Detailed Ledger             -> Chronological expense tracking
[3] Safe Revert System          -> Undo mistakes instantly
[4] Debt Priority Analytics      -> Real-time financial tracking

Team: Abdul Basit Shahid | Aizaz Noor | Muhammad Abdullah

-----

Press Enter to start..._
```

```
Command Prompt - .\HKM.exe

+-----+
|                                |
|          ROOMMATE BALANCES    |
| View live net balances for all members |
|                                |
+-----+

+-----+-----+-----+-----+
| ID      | Name      | Balance (Rs.) | Status  |
+-----+-----+-----+-----+
| 05      | safiyan   | Rs. 10000.00  | REMAINED |
| A01     | Ali       | Rs. 1000.00   | REMAINED |
| 01      | basit     | Rs. 2500.50   | REMAINED |
| 02      | haseeb    | Rs. 750.50    | REMAINED |
| 03      | aizaz     | Rs. 7500.00   | REMAINED |
+-----+-----+-----+-----+

Total Fund Cash Remaining : Rs. 21751.00

Press Enter to continue..._
```

Figure 3: Roommate balance output generated by MemberHash traversal.

```
Command Prompt - .\HKM.exe

+-----+
|              TRANSACTION TIMELINE              |
|  View all recorded expenses chronologically  |
+-----+

+-----+-----+-----+-----+
| # | Description          | Amount      | Paid By     |
+-----+-----+-----+-----+
| 1 | Final Refund (Clearance) | Rs. -10000.00 | FUND      |
| 2 | electricity            | Rs. 3500.00  | FUND      |
| 3 | Final Refund (Clearance) | Rs. -93250.00 | FUND      |
| 4 | pani ka bill           | Rs. 9999.00  | FUND      |
| 5 | nashta                 | Rs. 10000.00 | FUND      |
| 6 | Debt Settlement (Deposit) | Rs. 1000.00  | 04        |
+-----+-----+-----+-----+

Press Enter to continue...
```

Figure 4: Transaction timeline output generated through doubly linked list traversal.

```
Command Prompt - .\HKM.exe

+-----+
|              DEBT PRIORITY REPORT              |
|  Identify who owes the most money immediately  |
+-----+

Generated report for 0 debtors.

=====
DEBT SUMMARY REPORT
=====

Great news! No one owes any money.
=====

Press Enter to continue...
```

Figure 5: Debt priority report generated using max-heap extraction.

## 5.1 Saved Data Files Output

The project saves runtime state to CSV files so data remains available after closing and reopening the program.

### members.csv sample

```
05|safiyan|12345|10000
A01|Ali|password123|1000
01|basit|123|2500.5
02|haseeb|123|750.5
```

## transactions.csv sample

7

```
1|Final Refund (Clearance)|-10000|FUND|1|01|-10000
2|electricity|3500|FUND|2|01|-1750|02|-1750
3|Final Refund (Clearance)|-93250|FUND|1|01|-93250
4|pani ka bill|9999|FUND|2|01|-4999.5|02|-4999.5
5|nashta|10000|FUND|4|04|-2500|01|-2500|02|-2500|03|-2500
6|Debt Settlement (Deposit)|1000|04|1|04|1000
```

## 6. System Workflow

1. Program starts and configures Windows console for UTF-8 and ANSI colors.
2. Existing members.csv and transactions.csv are loaded into MemberHash and TransactionTimeline.
3. User enters the portal: admin login or roommate signup/login.
4. After login, the main menu offers member management, expense logging, undo, balances, reports, settlement, and save/exit.
5. When an expense is logged, splitExpense calculates balance changes and MemberHash updates member balances.
6. TransactionTimeline stores the transaction as a linked-list node and pushes its pointer onto the undo stack.
7. DebtHeap reads all members from MemberHash, filters negative balances, builds a heap, and prints highest debtors first.
8. Before logout and application close, saveData writes current members and transactions to CSV files.

## 7. Complexity Analysis

| Feature                | Data Structure     | Time Complexity               | Space Complexity            | Explanation                                                                  |
|------------------------|--------------------|-------------------------------|-----------------------------|------------------------------------------------------------------------------|
| Add member             | Hash Map           | Average $O(1)$ , worst $O(n)$ | $O(1)$ per member           | Direct bucket access; collision chain traversal only when needed.            |
| Search/login member    | Hash Map           | Average $O(1)$ , worst $O(n)$ | $O(1)$                      | Uses hash index then scans bucket chain.                                     |
| Remove member          | Hash Map chain     | Average $O(1)$ , worst $O(n)$ | $O(1)$                      | Find node, relink previous pointer, delete node.                             |
| Log transaction        | DLL + Stack        | $O(1)$ plus split $O(n)$      | $O(1)$ node + changes array | Tail append and stack push are direct. Splitting scans members.              |
| Undo transaction       | Stack + DLL        | $O(k)$                        | $O(1)$                      | $k$ is number of recorded balance changes to reverse; DLL unlink is $O(1)$ . |
| Print timeline         | Doubly Linked List | $O(t)$                        | $O(1)$                      | Traverses all $t$ transactions once.                                         |
| Print balances         | Hash Map scan      | $O(n)$                        | $O(1)$                      | Scans all buckets and member nodes.                                          |
| Build debt heap        | Max-Heap           | $O(n \log n)$                 | $O(n)$                      | Inserts each debtor into heap one by one.                                    |
| Extract highest debtor | Max-Heap           | $O(\log n)$                   | $O(1)$                      | Root removal followed by heapifyDown.                                        |
| Save/load data         | File traversal     | $O(n + t)$                    | $O(n + t)$                  | Reads/writes all members and transactions.                                   |

## 8. Conclusions and Findings

- The project successfully demonstrates multiple DSA concepts in one practical application.
- Hash Map improves member searching and login compared with linear search.
- Doubly Linked List is suitable for transaction history because it keeps chronological order and supports pointer-based deletion.
- Stack is the correct choice for undo because the latest transaction must be reversed first.
- Max-Heap is useful for ranking debtors because the highest debtor can be extracted efficiently.

- Manual memory management improved understanding of constructors, destructors, dynamic arrays, linked nodes, and pointer safety.
- CSV persistence made the program more practical because data remains available after closing the application.

## 9. Challenges Faced

| Challenge                       | How It Was Solved / Learning                                                                                       |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Collision handling in Hash Map  | Separate chaining was used so multiple members can exist in the same bucket.                                       |
| Undo balance reversal           | Every transaction stores a BalanceChange array, so undo can apply the negative of each change.                     |
| Memory leaks                    | Destructors delete dynamically allocated linked-list nodes, stack nodes, heap arrays, and balance-change arrays.   |
| Deleting from linked structures | MemberHash relinks chain nodes; TransactionTimeline handles only/head/tail/middle deletion cases.                  |
| Integrating multiple modules    | main.cpp connects MemberHash, TransactionTimeline, and DebtHeap through clean class methods.                       |
| Input validation                | cin.fail(), flushCin(), and guard conditions prevent invalid choices or invalid amounts from crashing the program. |
| Persistence format              | Pipe-separated CSV lines were used to avoid confusion with spaces in names or descriptions.                        |

## 10. Potential Future Enhancements

- Add graphical user interface using Qt or a web-based frontend.
- Add role-based access so admin and roommates have different permissions.
- Add date/time stamps to every transaction for better auditing.
- Add search transaction by ID, date, payer, or amount.
- Add redo functionality using a second stack.
- Optimize debt settlement by suggesting exact payments between debtors and creditors.
- Add password hashing instead of storing passwords directly in CSV.
- Export balance reports as PDF or Excel.
- Improve file error handling and backup recovery.
- Replace fixed heap capacity with a resizing dynamic array.

## 11. Contribution of Each Group Member

| Group Member       | Contribution                                                                                                                                       | Files / Modules                                        |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Aizaz Noor         | Designed and implemented transaction history and undo engine. Explained DLL, Stack, transaction nodes, undo deletion cases, and memory management. | TransactionTimeline.h, TransactionTimeline.cpp,        |
| Abdul Basit Shahid | Designed and implemented debt analytics, max-heap insertion/extraction, debtor ranking, and summary report.                                        | DebtHeap.h, DebtHeap.cpp, DebtHeap Functionality Guide |
| Muhammad Abdullah  | Designed and implemented member profile management, hash map, login support, balance updates, member printing, and member collection for heap.     | MemberHash.h, MemberHash.cpp                           |
| All Members        | Integrated modules in main.cpp, tested application features, prepared proposal/report, screenshots, and oral examination preparation.              | main.cpp, report, testing                              |

## Source Code: MemberHash.h

This appendix contains the complete code of MemberHash.h.

```
#ifndef MEMBERHASH_H
#define MEMBERHASH_H

#include <iostream>
#include <string>

using namespace std;

// Node for separate chaining in the hash map
struct MemberNode {
    string id;
    string name;
    string password;
    double balance; // Positive = owed money, Negative = owes money
    MemberNode* next;

    // Initialize member node data
    MemberNode(string _id, string _name, string _password, double _initialDeposit)
        : id(_id), name(_name), password(_password), balance(_initialDeposit), next(nullptr) {}
};

// Hash map storage for roommate profiles
class MemberHash {
private:
    static const int TABLE_SIZE = 100; // Total buckets in the hash table
    MemberNode* table[TABLE_SIZE];

    // Compute bucket index from ID by summing ASCII values
    int hashFunction(const string& id);

public:
    MemberHash();
    ~MemberHash(); // Destructor to free all allocated chain nodes

    // Add a new member. Returns true on success, false if ID exists.
    bool addMember(const string& id, const string& name, const string& password, double initialDeposit = 0.0);

    // Remove a member. Only allowed if balance is settled (0.0).
    bool removeMember(const string& id);

    // Find a member node by ID. Returns nullptr if not found.
    MemberNode* getMember(const string& id);

    // Verify member ID and password
    bool login(const string& id, const string& password);

    // Adjust balance of a member by the given amount
    void updateBalance(const string& id, double amount);

    // Print all registered members in a grid format
    void printAllMembers();

    // Collect all member pointers into a dynamic array for heap analytics
    MemberNode** getAllMembers(int& count);
};

#endif // MEMBERHASH_H
```

## Source Code: MemberHash.cpp

This appendix contains the complete code of MemberHash.cpp.

```
#include "../include/MemberHash.h"
#include <iomanip>

// Set all hash table buckets to nullptr
MemberHash::MemberHash() {
    for (int i = 0; i < TABLE_SIZE; ++i) {
        table[i] = nullptr;
    }
}

// Free all memory by deleting chained nodes
MemberHash::~MemberHash() {
    for (int i = 0; i < TABLE_SIZE; ++i) {
        MemberNode* current = table[i];
        while (current != nullptr) {
            MemberNode* temp = current->next;
            delete current;
            current = temp;
        }
        table[i] = nullptr;
    }
}

// Sum ASCII values of characters in ID and modulo by TABLE_SIZE
int MemberHash::hashFunction(const string& id) {
    int hash = 0;
    for (char c : id) {
        hash += (int)c;
    }
    return hash % TABLE_SIZE;
}

// Register a new member. Returns false if inputs are blank or ID is taken.
bool MemberHash::addMember(const string& id, const string& name, const string& password, double initialDeposit) {
    if (id.empty() || name.empty()) {
        return false;
    }

    int index = hashFunction(id);

    // Look for duplicate ID in the bucket chain
    MemberNode* current = table[index];
    while (current != nullptr) {
        if (current->id == id) {
            return false;
        }
        current = current->next;
    }

    // Insert new node at the head of the chain (constant time O(1))
    MemberNode* newNode = new MemberNode(id, name, password, initialDeposit);
    newNode->next = table[index];
    table[index] = newNode;
    return true;
}

// Delete a member node. Returns false if not found or if balance is not zero.
bool MemberHash::removeMember(const string& id) {
    if (id.empty()) return false;

    int index = hashFunction(id);
    MemberNode* current = table[index];
    MemberNode* prev = nullptr;

    while (current != nullptr) {
        if (current->id == id) {
            // Cannot delete if they still owe money or have a surplus

```

```

        if (current->balance != 0.0) {
            return false;
        }

        // Unlink node from chain
        if (prev == nullptr) {
            table[index] = current->next;
        } else {
            prev->next = current->next;
        }
        delete current;
        return true;
    }
    prev = current;
    current = current->next;
}
return false;
}

// Search for a member by ID. Returns pointer or nullptr.
MemberNode* MemberHash::getMember(const string& id) {
    int index = hashFunction(id);

    MemberNode* current = table[index];
    while (current != nullptr) {
        if (current->id == id) {
            return current;
        }
        current = current->next;
    }
    return nullptr;
}

// Validate credentials for portal entry
bool MemberHash::login(const string& id, const string& password) {
    MemberNode* member = getMember(id);
    if (member != nullptr && member->password == password) {
        return true;
    }
    return false;
}

// Add amount to the specified member's balance
void MemberHash::updateBalance(const string& id, double amount) {
    MemberNode* member = getMember(id);

    if (member == nullptr) {
        cout << " ERROR: Member ID '" << id << "' not found in Hash Map.\n";
        return;
    }

    member->balance += amount;
}

// Print all registered members in a grid
void MemberHash::printAllMembers() {
    bool anyMember = false;

    cout << "\n";
    cout << " +-----+-----+-----+-----+\n";
    cout << " | ID   | Name       | Balance (Rs.) | Status  |\n";
    cout << " +-----+-----+-----+-----+\n";

    double totalFundRemaining = 0.0;

    for (int i = 0; i < TABLE_SIZE; ++i) {
        MemberNode* current = table[i];
        while (current != nullptr) {
            anyMember = true;
            totalFundRemaining += current->balance;

```

```

string status;
if (current->balance > 0)    status = "REMAINED";
else if (current->balance < 0) status = "OWES";
else                        status = "SETTLED";

// Limit display name length to fit column formatting
string name = current->name;
if (name.length() > 18) name = name.substr(0, 15) + "...";

cout << " | "
    << left << setw(6) << current->id    << " | "
    << left << setw(18) << name           << " | Rs. "
    << right << setw(9) << fixed << setprecision(2) << current->balance << " | "
    << left << setw(8) << status         << " |\n";

    current = current->next;
}
}

if (!anyMember) {
    cout << " |          No members registered yet.          |\n";
}

cout << " +-----+-----+-----+-----+\n\n";

if (anyMember) {
    cout << " Total Fund Cash Remaining : Rs. "
        << fixed << setprecision(2) << totalFundRemaining << "\n";
}
}

// Build a dynamic array of all member pointers for external processing (like heap sorting)
MemberNode** MemberHash::getAllMembers(int& outCount) {
    int count = 0;
    for (int i = 0; i < TABLE_SIZE; i++) {
        MemberNode* curr = table[i];
        while (curr != nullptr) {
            count++;
            curr = curr->next;
        }
    }

    MemberNode** result = new MemberNode*[count];
    int index = 0;
    for (int i = 0; i < TABLE_SIZE; i++) {
        MemberNode* curr = table[i];
        while (curr != nullptr) {
            result[index++] = curr;
            curr = curr->next;
        }
    }

    outCount = count;
    return result;
}

```

## Source Code: TransactionTimeline.h

This appendix contains the complete code of TransactionTimeline.h.

```
#ifndef TRANSACTIONTIMELINE_H
#define TRANSACTIONTIMELINE_H

#include <iostream>
#include <string>

class MemberHash;

using namespace std;

// Structure to track balance updates for individuals during a split
struct BalanceChange {
    string memberId;
    double changeAmount;
};

// Node for the doubly linked transaction list
struct TransactionNode {
    int transactionId;
    string description;
    double amount;
    string payerId;

    BalanceChange* changes; // Array of updates made to individual balances
    int numChanges;        // Number of updates in the changes array

    TransactionNode* prev;
    TransactionNode* next;

    TransactionNode(int id, string desc, double amt, string payer, BalanceChange* chgs, int numChgs)
        : transactionId(id), description(desc), amount(amt), payerId(payer),
          changes(chgs), numChanges(numChgs), prev(NULL), next(NULL) {}

    ~TransactionNode() {
        if (changes != NULL) {
            delete[] changes;
        }
    }
};

// Node for the stack used to undo transactions
struct StackNode {
    TransactionNode* transactionRef; // Pointer to transaction node to undo
    StackNode* next;

    StackNode(TransactionNode* ref) : transactionRef(ref), next(NULL) {}
};

// Chronological transaction timeline manager
class TransactionTimeline {
private:
    TransactionNode* head;
    TransactionNode* tail;
    StackNode* undoStackTop;
    int transactionCounter;

public:
    TransactionTimeline();
    ~TransactionTimeline(); // Free both list and stack node memory

    // Add and split a new expense
    void logTransaction(string desc, double amount, string payerId, BalanceChange* changes, int numChanges);

    // Add transaction directly from file history
    void restoreTransaction(int id, string desc, double amount, string payerId, BalanceChange* changes, int numChanges);
};
```

```
// Revert the latest transaction
void undoLastTransaction(MemberHash* hash);

// Print all transactions in a list
void printTimeline();

TransactionNode* getHead() const { return head; }
int getCounter() const { return transactionCounter; }
void setCounter(int val) { transactionCounter = val; }
};

#endif // TRANSACTIONTIMELINE_H
```

## Source Code: TransactionTimeline.cpp

This appendix contains the complete code of TransactionTimeline.cpp.

```
#include "../include/TransactionTimeline.h"
#include "../include/MemberHash.h"
#include <iomanip>
#include <iostream>

// Initialize timeline fields
TransactionTimeline::TransactionTimeline() {
    head = nullptr;
    tail = nullptr;
    undoStackTop = nullptr;
    transactionCounter = 1;
}

// Free all memory in list and stack
TransactionTimeline::~TransactionTimeline() {
    TransactionNode* current = head;
    while (current != nullptr) {
        TransactionNode* temp = current->next;
        delete current;
        current = temp;
    }

    StackNode* stackCurrent = undoStackTop;
    while (stackCurrent != nullptr) {
        StackNode* temp = stackCurrent->next;
        delete stackCurrent;
        stackCurrent = temp;
    }
}

// Insert transaction directly into doubly linked list (used when loading saved files)
void TransactionTimeline::restoreTransaction(int id, string desc, double amount, string payerId, BalanceChange* changes, int numChanges) {
    TransactionNode* newNode = new TransactionNode(id, desc, amount, payerId, changes, numChanges);
    if (head == nullptr) {
        head = newNode;
        tail = newNode;
    } else {
        newNode->prev = tail;
        tail->next = newNode;
        tail = newNode;
    }
}

// Add a transaction node to the end of list and push it onto the undo stack
void TransactionTimeline::logTransaction(string desc, double amount, string payerId, BalanceChange* changes, int numChanges) {
    TransactionNode* newNode = new TransactionNode(transactionCounter, desc, amount, payerId, changes, numChanges);
    transactionCounter++;

    if (head == nullptr) {
        head = newNode;
        tail = newNode;
    } else {
        newNode->prev = tail;
        tail->next = newNode;
        tail = newNode;
    }

    // Push transaction node onto stack
    StackNode* stackNode = new StackNode(newNode);
    stackNode->next = undoStackTop;
    undoStackTop = stackNode;

    cout << " Expense " << desc << " (Rs. " << amount << ") logged as Transaction #" << newNode->transactionId << ".\n";
}

// Pop latest transaction from stack, reverse balance changes, and remove it from list
void TransactionTimeline::undoLastTransaction(MemberHash* hash) {
    if (undoStackTop == nullptr) {
```

```

    cout << " Nothing to undo — No transactions recorded yet!\n";
    return;
}

// Pop the top of stack
StackNode* topStack = undoStackTop;
TransactionNode* toRemove = topStack->transactionRef;
undoStackTop = undoStackTop->next;
delete topStack;

cout << " Undoing Transaction #" << toRemove->transactionId
    << " - " << toRemove->description << "\n";

// Reverse changes applied to member balances
if (hash != nullptr && toRemove->changes != nullptr) {
    for (int i = 0; i < toRemove->numChanges; i++) {
        // Apply negative of the change amount to reset
        hash->updateBalance(toRemove->changes[i].memberId, -(toRemove->changes[i].changeAmount));
    }
    cout << " Successfully reversed balance changes.\n";
}

// Remove the node from the doubly linked list
if (toRemove->prev == nullptr && toRemove->next == nullptr) {
    // Only one node in list
    head = nullptr;
    tail = nullptr;
}
else if (toRemove->prev == nullptr) {
    // Removing head node
    head = toRemove->next;
    head->prev = nullptr;
}
else if (toRemove->next == nullptr) {
    // Removing tail node
    tail = toRemove->prev;
    tail->next = nullptr;
}
else {
    // Removing middle node
    toRemove->prev->next = toRemove->next;
    toRemove->next->prev = toRemove->prev;
}

delete toRemove;
cout << " Undo complete!\n";
}

// Print transaction records from head to tail
void TransactionTimeline::printTimeline() {
    if (head == nullptr) {
        cout << "\n No transactions recorded yet.\n";
        return;
    }

    cout << "\n";
    cout << " +---+-----+-----+-----+\n";
    cout << " | # | Description          | Amount   | Paid By   |\n";
    cout << " +---+-----+-----+-----+\n";

    TransactionNode* current = head;
    while (current != nullptr) {
        string desc = current->description;
        string payer = current->payerId;

        // Truncate values if they exceed column widths
        if (desc.length() > 26) desc = desc.substr(0, 23) + "...";
        if (payer.length() > 12) payer = payer.substr(0, 12);

        cout << " | "
            << setw(2) << left << current->transactionId << " | "
            << setw(26) << left << desc << " | Rs. "

```

```
<< setw(7) << right << fixed << setprecision(2) << current->amount << " | "  
<< setw(12) << left << payer << " |\n";
```

```
current = current->next;  
}
```

```
cout << " +---+-----+-----+-----+\n";  
}
```

## Source Code: DebtHeap.h

This appendix contains the complete code of DebtHeap.h.

```
#ifndef DEBTHEAP_H
#define DEBTHEAP_H

#include <iostream>
#include <string>

using namespace std;

class MemberHash;
struct MemberNode;
extern MemberHash* globalMemberHash;

// Structure for tracking a roommate's debt amount
struct DebtRecord {
    string memberId;
    string memberName;
    double amountOwed; // Positive value representing the amount the roommate owes

    DebtRecord() : memberId(""), memberName(""), amountOwed(0.0) {}
    DebtRecord(string id, string name, double amt) : memberId(id), memberName(name), amountOwed(amt) {}
};

// Max-heap to sort debtors by amount owed
class DebtHeap {
private:
    DebtRecord* heapArray; // Array for binary heap tree
    int capacity;
    int size;

    // Helper functions for binary heap indices
    int parent(int i) { return (i - 1) / 2; }
    int leftChild(int i) { return (2 * i) + 1; }
    int rightChild(int i) { return (2 * i) + 2; }

    // Shift elements up/down to keep heap properties intact
    void heapifyUp(int index);
    void heapifyDown(int index);

public:
    DebtHeap(int cap);
    ~DebtHeap();

    // Disable copying and assignment to avoid memory leaks
    DebtHeap(const DebtHeap&) = delete;
    DebtHeap& operator=(const DebtHeap&) = delete;

    // Insert a debt entry
    void insertDebt(string id, string name, double amount);

    // Extract and return the largest debtor
    DebtRecord extractMaxDebtor();

    // Read balances from MemberHash and build the heap
    void buildHeapFromHashMap();

    // Print all debtors sorted by size of debt
    void printSummaryReport();
};

#endif // DEBTHEAP_H
```

## Source Code: DebtHeap.cpp

This appendix contains the complete code of DebtHeap.cpp.

```
#include "../include/DebtHeap.h"
#include "../include/MemberHash.h"

DebtHeap::DebtHeap(int cap) {
    capacity = cap;
    size = 0;
    heapArray = new DebtRecord[capacity];
}

DebtHeap::~DebtHeap() {
    delete[] heapArray;
}

// Push the element up the tree if it is larger than its parent
void DebtHeap::heapifyUp(int index) {
    while (index > 0) {
        int parentIdx = parent(index);

        if (heapArray[index].amountOwed > heapArray[parentIdx].amountOwed) {
            DebtRecord temp = heapArray[index];
            heapArray[index] = heapArray[parentIdx];
            heapArray[parentIdx] = temp;

            index = parentIdx;
        } else {
            break;
        }
    }
}

// Push the element down the tree to its correct spot
void DebtHeap::heapifyDown(int index) {
    while (true) {
        int leftIdx = leftChild(index);
        int rightIdx = rightChild(index);
        int largestIdx = index;

        if (leftIdx < size &&
            heapArray[leftIdx].amountOwed > heapArray[largestIdx].amountOwed) {
            largestIdx = leftIdx;
        }

        if (rightIdx < size &&
            heapArray[rightIdx].amountOwed > heapArray[largestIdx].amountOwed) {
            largestIdx = rightIdx;
        }

        if (largestIdx != index) {
            DebtRecord temp = heapArray[index];
            heapArray[index] = heapArray[largestIdx];
            heapArray[largestIdx] = temp;

            index = largestIdx;
        } else {
            break;
        }
    }
}

// Add a debtor record and restore heap property
void DebtHeap::insertDebt(string id, string name, double amount) {
    if (size >= capacity) {
        cout << " ! Maximum limit reached! Cannot process more debtors.\n";
        return;
    }
}
```

```

    heapArray[size] = DebtRecord(id, name, amount);
    size++;
    heapifyUp(size - 1);
}

// Remove and return the debtor who owes the most
DebtRecord DebtHeap::extractMaxDebtor() {
    if (size <= 0) {
        cout << " ! No debtors found.\n";
        return DebtRecord();
    }

    DebtRecord maxDebtor = heapArray[0];
    heapArray[0] = heapArray[size - 1];
    size--;

    if (size > 0) {
        heapifyDown(0);
    }

    return maxDebtor;
}

// Scan the roommate hash map and build a heap from members with negative balances
void DebtHeap::buildHeapFromHashMap() {
    size = 0;

    if (globalMemberHash == nullptr) {
        cout << " ! System not initialized! Cannot generate report.\n";
        return;
    }

    int count = 0;
    MemberNode** allMembers = globalMemberHash->getAllMembers(count);

    for (int i = 0; i < count; i++) {
        MemberNode* member = allMembers[i];
        if (member != nullptr && member->balance < 0) {
            double amountOwed = -(member->balance);
            insertDebt(member->id, member->name, amountOwed);
        }
    }

    delete[] allMembers;
    cout << " Generated report for " << size << " debtors.\n";
}

// Extract all debtors from the heap one by one to print a sorted list
void DebtHeap::printSummaryReport() {
    cout << "\n" << string(50, '=') << "\n";
    cout << " DEBT SUMMARY REPORT\n";
    cout << string(50, '=') << "\n\n";

    if (size == 0) {
        cout << "Great news! No one owes any money.\n";
        cout << string(50, '=') << "\n\n";
        return;
    }

    cout << "Members sorted by amount owed (highest first):\n\n";

    int rank = 1;
    double totalDebt = 0;

    while (size > 0) {
        DebtRecord debtor = extractMaxDebtor();

        cout << rank << ". " << debtor.memberName
            << " (" << debtor.memberId << "): "

```

```
<< "Rs. " << debtor.amountOwed << "\n";
```

```
totalDebt += debtor.amountOwed;
```

```
rank++;
```

```
}
```

```
cout << "\n" << string(50, '=') << "\n";
```

```
cout << "Total Amount Owed: Rs. " << totalDebt << "\n";
```

```
cout << string(50, '=') << "\n\n";
```

```
}
```

## Source Code: main.cpp

This appendix contains the complete code of main.cpp.

```
#include <iostream>
#include <string>
#include <iomanip>
#include <sstream>
#include <limits>
#include <fstream>
#include <windows.h>
#include "../include/MemberHash.h"
#include "../include/TransactionTimeline.h"
#include "../include/DebtHeap.h"

MemberHash* globalMemberHash = nullptr;

using namespace std;

// Configure Windows terminal to support UTF-8 and ANSI colors
void setupConsole() {
    SetConsoleOutputCP(65001);
    SetConsoleCP(65001);
    HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD dwMode = 0;
    GetConsoleMode(hOut, &dwMode);
    SetConsoleMode(hOut, dwMode | ENABLE_VIRTUAL_TERMINAL_PROCESSING);
}

// ANSI color codes (prefixed with C_ to avoid Windows header name conflicts)
const string C_RST = "\033[0m";
const string C_BD = "\033[1m";
const string C_DIM = "\033[2m";
const string C_GRN = "\033[92m"; // bright green
const string C_YLW = "\033[93m"; // bright yellow
const string C_RED = "\033[91m"; // bright red
const string C_CYN = "\033[96m"; // bright cyan
const string C_BLU = "\033[94m"; // bright blue
const string C_MGT = "\033[95m"; // bright magenta
const string C_WHT = "\033[97m"; // white
const string C_GRY = "\033[90m"; // dark grey

// UI box panel width in characters
const int W = 54;

// Print box layout line dividers
void divTop() { cout << C_CYN << " +" << string(W, '-') << "+" << C_RST << "\n"; }
void divMid() { cout << C_CYN << " |" << string(W, '-') << "|" << C_RST << "\n"; }
void divBot() { cout << C_CYN << " +" << string(W, '-') << "+" << C_RST << "\n"; }
void divLine() { cout << C_GRY << " " << string(W + 2, '-') << C_RST << "\n"; }

// Print a string centered inside the UI panel
// visLen specifies the length of text without ANSI color codes
void centerRow(const string& text, int visLen, const string& colorPrefix = "") {
    int pad = W - visLen;
    int left = pad / 2;
    int right = pad - left;
    cout << C_CYN << " |" << C_RST
        << string(left, ' ')
        << colorPrefix << text << C_RST
        << string(right, ' ')
        << C_CYN << "|" << C_RST << "\n";
}

// Print a formatted menu option row
void menuRow(const string& num, const string& label, const string& tag,
            const string& numColor, const string& tagColor) {
    // Visible layout inside the | borders:
    // " [ N ] <label padded to 30> [tag padded to 10] " = 52 visible chars = W-2
    string visNum = "[" + num + " ]"; // 5 chars
    int labelW = 31;
    int tagW = 12;
```

```

string tagStr = tag.empty() ? string(tagW, ' ') : ("[" + tag + "]");
int tagPad  = tagW - (int)tagStr.size();
if (tagPad < 0) tagPad = 0;

int labelPad = labelW - (int)label.size();
if (labelPad < 1) labelPad = 1;

// visible width of content = 2 + 5 + 2 + label (31) + tagStr (12) + 2
// = 9 + 31 + 12 + 2 = 54 = W
cout << C_CYN << " |" << C_RST << " "
    << numColor << C_BD << visNum << C_RST << " "
    << C_WHT << label << C_RST
    << string(labelPad, ' ')
    << tagColor << C_DIM << tagStr << C_RST
    << string(tagPad, ' ') << " "
    << C_CYN << "|" << C_RST << "\n";
}

// General UI helper functions
void clearScreen() { system("cls"); }

void flushCin() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

void pause() {
    cout << "\n" << C_DIM << C_CYN << " Press Enter to continue..." << C_RST;
    string dummy;
    getline(cin, dummy);
}

// Calculate the hash map bucket index for an ID
int computeBucket(const string& id) {
    int h = 0;
    for (char c : id) h += (int)c;
    return h % 100;
}

// Display terminal splash screen with system overview
void printSplash() {
    clearScreen();
    cout << "\n";
    cout << C_CYN << "  _ _ _ _ _  \n";
    cout << "   / / /  / / _ /  / / \n";
    cout << "  / _ /  / , <  / / _ / \n";
    cout << "   / / _ /  / _ /  / / / \n" << C_RST;
    cout << "\n";
    cout << C_BD << C_WHT << " H O S T E L   K H A R C H A   M A N A G E R\n" << C_RST;
    cout << "\n";
    cout << C_GRY;
    cout << "  System Features:\n";
    cout << "   [1] Fast Member Registration   -> Manage profiles & deposits\n";
    cout << "   [2] Detailed Ledger           -> Chronological expense tracking\n";
    cout << "   [3] Safe Revert System        -> Undo mistakes instantly\n";
    cout << "   [4] Debt Priority Analytics    -> Real-time financial tracking\n";
    cout << C_RST << "\n";
    cout << C_DIM << C_GRY << "  Team: Basit Shahid | Aizaz Noor | Abdul\n" << C_RST;
    cout << "\n";
    divLine();
    cout << "\n" << C_DIM << C_CYN << " Press Enter to start..." << C_RST;
    cin.get();
}

// Print header block above menus showing current counts
void printHeader(int members, int txns) {
    string title  = "HOSTEL KHARCHA MANAGER";
    string subtitle = "Members: " + to_string(members)
        + " | Transactions: " + to_string(txns);
    cout << "\n";
    divTop();
}

```

```

        centerRow(title, (int)title.size(), C_BD + C_WHT);
        centerRow(subtitle, (int)subtitle.size(), C_DIM + C_GRY);
        divBot();
    }
}

```

```

// Display the primary main menu options
void printMenu(int members, int txns) {
    clearScreen();
    printHeader(members, txns);
    cout << "\n";
    divTop();
    centerRow("MAIN MENU", 9, C_BD + C_CYN);
    divMid();
    menuRow("1", "Add New Roommate", "Profile", C_GRN, C_GRN);
    menuRow("2", "Remove a Roommate", "Delete", C_RED, C_RED);
    menuRow("3", "Log a Shared Expense", "Ledger", C_YLW, C_YLW);
    menuRow("4", "Undo Last Transaction", "Revert", C_RED, C_RED);
    menuRow("5", "View Transaction Timeline", "History", C_CYN, C_CYN);
    menuRow("6", "View Roommate Balances", "Finances", C_BLU, C_BLU);
    menuRow("7", "Who Owes The Most?", "Analytics", C_MGT, C_MGT);
    menuRow("8", "Settle Debt / Deposit", "Clear", C_GRN, C_GRN);
    menuRow("9", "Exit & Save Data", "", C_GRY, C_GRY);
    divBot();
    cout << "\n " << C_BD << "Your choice: " << C_RST;
}

```

```

// Print a sub-header block when navigating to a feature section
void sectionHeader(const string& title, const string& subtitle, const string& color) {
    cout << "\n";
    divTop();
    centerRow(title, (int)title.size(), C_BD + color);
    centerRow(subtitle, (int)subtitle.size(), C_DIM + C_GRY);
    divBot();
    cout << "\n";
}

```

```

// Split an expense using either equal or custom weighted sharing
void splitExpense(MemberHash& profiles, const string& payerId, double amount, BalanceChange*& outChanges, int& outNumChanges) {
    int count = 0;
    MemberNode** members = profiles.getAllMembers(count);
    if (count == 0) {
        delete[] members;
        outChanges = nullptr;
        outNumChanges = 0;
        return;
    }
}

```

```

    outNumChanges = count;
    outChanges = new BalanceChange[count];
    int changeldx = 0;

```

```

    cout << "\n How to split this expense?\n";
    cout << " 1. Equal Split\n";
    cout << " 2. Custom Weighted Distribution\n";
    cout << " Choice: ";
    int splitChoice;
    cin >> splitChoice;

```

```

    if (cin.fail() || (splitChoice != 1 && splitChoice != 2)) {
        flushCin();
        cout << C_RED << " ! Invalid choice. Falling back to Equal Split.\n" << C_RST;
        splitChoice = 1;
    } else {
        flushCin();
    }
}

```

```

    if (splitChoice == 1) {
        double share = amount / count;

```

```

        if (payerId == "FUND") {
            for (int i = 0; i < count; i++) {
                profiles.updateBalance(members[i]->id, -share);
            }
        }
    }
}

```

```

        outChanges[changeldx++] = {members[i]->id, -share};
    }
    cout << C_GRY
        << "\n Per-person share : Rs. " << fixed << setprecision(2) << share
        << "\n Paid by Common Fund (No individual credited)"
        << "\n" << C_RST;
} else {
    // Payer gets back (amount - their own share)
    profiles.updateBalance(payerId, amount - share);
    outChanges[changeldx++] = {payerId, amount - share};

    // Everyone else is debited their share
    for (int i = 0; i < count; i++) {
        MemberNode* m = members[i];
        if (m->id != payerId) {
            profiles.updateBalance(m->id, -share);
            outChanges[changeldx++] = {m->id, -share};
        }
    }
}

cout << C_GRY
    << "\n Per-person share : Rs. " << fixed << setprecision(2) << share
    << "\n Payer credited : Rs. " << fixed << setprecision(2) << (amount - share)
    << "\n" << C_RST;
}
} else {
    double totalWeighted = 0;
    double* weights = new double[count];

    cout << "\n Enter weight or amount for each member:\n";
    for (int i = 0; i < count; i++) {
        MemberNode* m = members[i];
        cout << " " << m->name << " (" << m->id << "): ";
        cin >> weights[i];
        if (cin.fail() || weights[i] < 0) {
            flushCin();
            cout << C_RED << " ! Invalid weight. Defaulting to 0.\n" << C_RST;
            weights[i] = 0;
        } else {
            flushCin();
        }
        totalWeighted += weights[i];
    }

    if (totalWeighted == 0) {
        cout << C_RED << " ! Total weight cannot be zero. Falling back to Equal Split.\n" << C_RST;
        double share = amount / count;

        if (payerId == "FUND") {
            for (int i = 0; i < count; i++) {
                profiles.updateBalance(members[i]->id, -share);
                outChanges[changeldx++] = {members[i]->id, -share};
            }
        } else {
            profiles.updateBalance(payerId, amount - share);
            outChanges[changeldx++] = {payerId, amount - share};
            for (int i = 0; i < count; i++) {
                MemberNode* m = members[i];
                if (m->id != payerId) {
                    profiles.updateBalance(m->id, -share);
                    outChanges[changeldx++] = {m->id, -share};
                }
            }
        }
    } else {
        // If paid by a specific member, credit them first
        if (payerId != "FUND") {
            profiles.updateBalance(payerId, amount);
        }

        // Then deduct everyone's share based on weight
        for (int i = 0; i < count; i++) {
            MemberNode* m = members[i];

```

```

        double share = (weights[i] / totalWeighted) * amount;
        profiles.updateBalance(m->id, -share);

        cout << C_GRY << " - " << m->name << " share: Rs. "
             << fixed << setprecision(2) << share << C_RST << "\n";
    }
}
delete[] weights;
}

delete[] members;
}

// Save current data state to CSV files
void saveData(MemberHash& profiles, TransactionTimeline& timeline) {
    ofstream mf("members.csv");
    int count = 0;
    MemberNode** members = profiles.getAllMembers(count);
    for (int i = 0; i < count; i++) {
        mf << members[i]->id << "|" << members[i]->name << "|" << members[i]->password << "|" << members[i]->balance << "\n";
    }
    delete[] members;
    mf.close();

    ofstream tf("transactions.csv");
    tf << timeline.getCounter() << "\n"; // Save the counter to prevent ID conflicts
    TransactionNode* current = timeline.getHead();
    while (current != nullptr) {
        tf << current->transactionId << "|" << current->description << "|"
          << current->amount << "|" << current->payerId << "|" << current->numChanges;
        for (int i = 0; i < current->numChanges; i++) {
            tf << "|" << current->changes[i].memberId << "|" << current->changes[i].changeAmount;
        }
        tf << "\n";
        current = current->next;
    }
    tf.close();
}

void loadData(MemberHash& profiles, TransactionTimeline& timeline, int& membersCount, int& txnsCount) {
    ifstream mf("members.csv");
    if (mf.is_open()) {
        string line;
        while (getline(mf, line)) {
            stringstream ss(line);
            string id, name, part3, part4;
            if (getline(ss, id, '|') && getline(ss, name, '|')) {
                string password = "123"; // default password
                double bal = 0.0;
                if (getline(ss, part3, '|')) {
                    if (getline(ss, part4, '|')) {
                        // formats: id|name|password|balance
                        password = part3;
                        bal = stod(part4);
                    } else {
                        // formats: id|name|balance (old format)
                        bal = stod(part3);
                    }
                }
            }
            if (profiles.addMember(id, name, password, 0)) { // Add with 0 initial deposit
                profiles.updateBalance(id, bal); // Sync to exact balance
                membersCount++;
            }
        }
    }
    mf.close();
}

ifstream tf("transactions.csv");
if (tf.is_open()) {
    string counterStr;
    if (getline(tf, counterStr)) {
        timeline.setCounter(stoi(counterStr));
    }
}

```

```

    }
    string line;
    while (getline(tf, line)) {
        stringstream ss(line);
        string tld, desc, amtStr, payerId, numChgStr;
        if (getline(ss, tld, '|') && getline(ss, desc, '|') &&
            getline(ss, amtStr, '|') && getline(ss, payerId, '|') && getline(ss, numChgStr, '|')) {

            int id = stoi(tld);
            double amt = stod(amtStr);
            int numChg = stoi(numChgStr);

            BalanceChange* changes = nullptr;
            if (numChg > 0) {
                changes = new BalanceChange[numChg];
                for (int i = 0; i < numChg; i++) {
                    string cld, cAmt;
                    getline(ss, cld, '|');
                    getline(ss, cAmt, '|');
                    changes[i].memberId = cld;
                    changes[i].changeAmount = stod(cAmt);
                }
            }
            timeline.restoreTransaction(id, desc, amt, payerId, changes, numChg);
            txnsCount++;
        }
    }
    tf.close();
}
}

```

---

```

int main() {
    setupConsole();
    printSplash();

    MemberHash    profiles;
    globalMemberHash = &profiles;
    TransactionTimeline timeline;
    DebtHeap      analytics(100);

    int members = 0, txns = 0;

    loadData(profiles, timeline, members, txns);

    bool portalRunning = true;
    while (portalRunning) {
        clearScreen();
        cout << "\n\n";
        divTop();
        centerRow("HOSTEL KHARCHA MANAGER PORTAL", 29, C_CYN + C_BD);
        divMid();
        menuRow("1", "Log In (Roommate or Admin)", "Access System", C_GRN, C_GRN);
        menuRow("2", "Sign Up (Register Roommate)", "Join System", C_YLW, C_YLW);
        menuRow("3", "Exit Portal", "Quit App", C_RED, C_RED);
        divBot();
        cout << "\n " << C_BD << "Your choice: " << C_RST;

        int portalChoice = 0;
        cin >> portalChoice;
        if (cin.fail()) {
            flushCin();
            cout << C_RED << "\n ! Invalid option. Enter 1-3.\n" << C_RST;
            pause();
            continue;
        }
        flushCin();

        if (portalChoice == 1) {
            clearScreen();
            cout << "\n\n";
            divTop();

```

```

centerRow("ROOMMATE / ADMIN LOGIN", 22, C_CYN + C_BD);
divBot();
cout << "\n";

string user, pass;
cout << " Username / Member ID : ";
cin >> user;
flushCin();

cout << " Password      : ";
cin >> pass;
flushCin();

bool loginSuccess = false;
string displayName = "";

if (user == "admin" && pass == "admin123") {
    loginSuccess = true;
    displayName = "Administrator";
} else {
    MemberNode* m = profiles.getMember(user);
    if (m != nullptr && m->password == pass) {
        loginSuccess = true;
        displayName = m->name;
    }
}

if (loginSuccess) {
    cout << C_GRN << "\n Access Granted! Welcome, " << displayName << ".\n" << C_RST;
    pause();

    int choice = 0;
    while (choice != 9) {
        printMenu(members, txns);
        cin >> choice;

        if (cin.fail()) {
            flushCin();
            cout << C_RED << "\n ! Invalid option. Please enter a number.\n" << C_RST;
            pause();
            continue;
        }
        flushCin();

        // [1] Add Roommate
        if (choice == 1) {
            clearScreen();
            sectionHeader("ADD NEW ROOMMATE", "Register a new member to the hostel pool", C_GRN);

            string id, name, roommatePass;
            double deposit = 0.0;
            cout << " Enter Member ID (e.g. A01) : "; cin >> id;
            flushCin();
            cout << " Enter Member Name      : "; getline(cin, name);
            cout << " Enter Password      : "; cin >> roommatePass;
            flushCin();
            cout << " Initial Deposit (Rs.) : "; cin >> deposit;
            if (cin.fail() || deposit < 0) {
                flushCin();
                cout << C_RED << " ! Invalid deposit. Defaulting to 0.\n" << C_RST;
                deposit = 0.0;
            } else {
                flushCin();
            }
        }

        if (profiles.addMember(id, name, roommatePass, deposit)) {
            members++;
            cout << C_GRN << "\n + " << name << " (ID: " << id << ") added!\n" << C_RST;
            saveData(profiles, timeline);
        } else {
            cout << C_RED << "\n ! ID " << id << " already exists or input is empty.\n" << C_RST;
        }
    }
}

```

```

    pause();
}

// [2] Remove Roommate
else if (choice == 2) {
    clearScreen();
    sectionHeader("REMOVE A ROOMMATE", "Safely delete a member from the hostel pool", C_RED);

    if (members == 0) {
        cout << C_RED << " ! No members to remove.\n" << C_RST;
        pause();
        continue;
    }

    string id;
    cout << " Enter Member ID to remove: "; cin >> id;
    flushCin();

    MemberNode* m = profiles.getMember(id);
    if (m == nullptr) {
        cout << C_RED << "\n ! Member ID " << id << " not found.\n" << C_RST;
    } else {
        bool proceedWithDeletion = false;

        if (m->balance > 0.0) {
            cout << C_YLW << "\n ! " << m->name << " has a surplus of Rs. " << fixed << setprecision(2) << m->balance << "\n" << C_RST;
            cout << " Do you want to issue a refund and remove them? (Y/N): ";
            char confirm;
            cin >> confirm;
            flushCin();
            if (toupper(confirm) == 'Y') {
                double refundAmt = m->balance;
                profiles.updateBalance(id, -refundAmt); // Bring to 0

                BalanceChange* changes = new BalanceChange[1];
                changes[0].memberId = id;
                changes[0].changeAmount = -refundAmt;
                timeline.logTransaction("Final Refund (Clearance)", -refundAmt, "FUND", changes, 1);
                txns++;
                cout << C_GRN << " + Refund issued.\n" << C_RST;
                proceedWithDeletion = true;
            }
        } else if (m->balance < 0.0) {
            cout << C_RED << "\n ! " << m->name << " owes Rs. " << fixed << setprecision(2) << -m->balance << "\n" << C_RST;
            cout << " Has this debt been settled in cash? (Y/N): ";
            char confirm;
            cin >> confirm;
            flushCin();
            if (toupper(confirm) == 'Y') {
                double paymentAmt = -m->balance;
                profiles.updateBalance(id, paymentAmt); // Bring to 0

                BalanceChange* changes = new BalanceChange[1];
                changes[0].memberId = id;
                changes[0].changeAmount = paymentAmt;
                timeline.logTransaction("Final Settlement (Clearance)", paymentAmt, id, changes, 1);
                txns++;
                cout << C_GRN << " + Debt cleared.\n" << C_RST;
                proceedWithDeletion = true;
            }
        } else {
            cout << C_YLW << "\n Are you sure you want to remove " << m->name << "? (Y/N): " << C_RST;
            char confirm;
            cin >> confirm;
            flushCin();
            if (toupper(confirm) == 'Y') {
                proceedWithDeletion = true;
            }
        }
    }

    if (proceedWithDeletion) {
        if (profiles.removeMember(id)) {

```

```

        members--;
        cout << C_GRN << "\n + Member " << id << " removed successfully.\n" << C_RST;
    }
} else {
    cout << C_RED << "\n ! Deletion cancelled.\n" << C_RST;
}
}
pause();
}

// [3] Log Expense
else if (choice == 3) {
    clearScreen();
    sectionHeader("LOG SHARED EXPENSE", "Record a new expense and split it among members", C_YLW);

    if (members == 0) {
        cout << C_RED << " ! No members yet. Add roommates first (Option 1).\n" << C_RST;
        pause();
        continue;
    }

    string payerId, desc;
    double amount;

    cout << " Who paid? (Member ID or type 'FUND') : "; cin >> payerId;
    flushCin();

    for (char &c : payerId) c = toupper(c);

    if (payerId != "FUND" && !profiles.getMember(payerId)) {
        cout << C_RED << "\n ! Member ID " << payerId << " not found.\n" << C_RST;
        pause();
        continue;
    }

    cout << " What was it for?   : "; getline(cin, desc);
    cout << " Amount (Rs.)       : "; cin >> amount;
    if (cin.fail() || amount <= 0) {
        flushCin();
        cout << C_RED << "\n ! Invalid amount. Must be a positive number.\n" << C_RST;
        pause();
        continue;
    }
    flushCin();

    BalanceChange* changes = nullptr;
    int numChanges = 0;

    splitExpense(profiles, payerId, amount, changes, numChanges);

    timeline.logTransaction(desc, amount, payerId, changes, numChanges);
    txns++;

    cout << C_GRN << "\n + Logged and split among " << members << " members.\n" << C_RST;
    pause();
}

// [4] Undo Transaction
else if (choice == 4) {
    clearScreen();
    sectionHeader("UNDO LAST TRANSACTION", "Revert the most recent expense completely", C_RED);
    timeline.undoLastTransaction(globalMemberHash);
    if (txns > 0) txns--;
    pause();
}

// [5] Timeline
else if (choice == 5) {
    clearScreen();
    sectionHeader("TRANSACTION TIMELINE", "View all recorded expenses chronologically", C_CYN);
    timeline.printTimeline();
    pause();
}

```

```

}

// [6] Balances
else if (choice == 6) {
    clearScreen();
    sectionHeader("ROOMMATE BALANCES", "View live net balances for all members", C_BLU);
    profiles.printAllMembers();
    pause();
}

// [7] Debt Report
else if (choice == 7) {
    clearScreen();
    sectionHeader("DEBT PRIORITY REPORT", "Identify who owes the most money immediately", C_MGT);
    analytics.buildHeapFromHashMap();
    analytics.printSummaryReport();
    pause();
}

// [8] Settle Debt / Deposit
else if (choice == 8) {
    clearScreen();
    sectionHeader("SETTLE DEBT / DEPOSIT", "Add money to a roommate's account to clear debts", C_GRN);

    if (members == 0) {
        cout << C_RED << " ! No members yet. Add roommates first.\n" << C_RST;
        pause();
        continue;
    }

    string id;
    double amount;
    cout << " Enter Member ID (who is paying?): "; cin >> id;
    flushCin();

    MemberNode* m = profiles.getMember(id);
    if (m == nullptr) {
        cout << C_RED << "\n ! Member ID " << id << " not found.\n" << C_RST;
        pause();
        continue;
    }

    cout << " Enter Amount (Rs.) (+ for Deposit, - for Withdraw) : "; cin >> amount;
    if (cin.fail() || amount == 0) {
        flushCin();
        cout << C_RED << "\n ! Invalid amount. Must be non-zero.\n" << C_RST;
        pause();
        continue;
    }
    flushCin();

    profiles.updateBalance(id, amount);

    // Log as a transaction to allow undo
    BalanceChange* changes = new BalanceChange[1];
    changes[0].memberId = id;
    changes[0].changeAmount = amount;
    string tDesc = (amount > 0) ? "Debt Settlement (Deposit)" : "Refund (Withdrawal)";
    timeline.logTransaction(tDesc, amount, (amount > 0) ? id : "FUND", changes, 1);
    txns++;

    if (amount > 0) {
        cout << C_GRN << "\n + Rs. " << fixed << setprecision(2) << amount
            << " deposited for " << m->name << ".\n" << C_RST;
    } else {
        cout << C_GRN << "\n + Rs. " << fixed << setprecision(2) << -amount
            << " withdrawn/refunded for " << m->name << ".\n" << C_RST;
    }
    pause();
}

```

```

// [9] Logout / Exit menu
else if (choice == 9) {
    clearScreen();
    cout << "\n";
    divTop();
    centerRow("Logging Out & Saving Data...", 28, C_YLW);
    divBot();
    cout << "\n";
    saveData(profiles, timeline);
    pause();
}

else {
    cout << C_RED << "\n ! Invalid option. Enter 1-9.\n" << C_RST;
    pause();
}
}
} else {
    cout << C_RED << "\n Invalid Credentials! Access Denied.\n" << C_RST;
    pause();
}
}

else if (portalChoice == 2) {
    clearScreen();
    cout << "\n\n";
    divTop();
    centerRow("ROOMMATE REGISTRATION / SIGN UP", 30, C_YLW + C_BD);
    divBot();
    cout << "\n";

    string id, name, roommatePass;
    double deposit = 0.0;
    cout << " Enter Member ID (e.g. A01) : "; cin >> id;
    flushCin();
    cout << " Enter Member Name : "; getline(cin, name);
    cout << " Enter Password : "; cin >> roommatePass;
    flushCin();
    cout << " Initial Deposit (Rs.) : "; cin >> deposit;
    if (cin.fail() || deposit < 0) {
        flushCin();
        cout << C_RED << " ! Invalid deposit. Defaulting to 0.\n" << C_RST;
        deposit = 0.0;
    } else {
        flushCin();
    }
}

if (profiles.addMember(id, name, roommatePass, deposit)) {
    members++;
    cout << C_GRN << "\n + Roommate " << name << " registered successfully!\n" << C_RST;
    cout << " You can now log in using ID: " << id << "\n";
    saveData(profiles, timeline);
} else {
    cout << C_RED << "\n ! ID " << id << " already exists or input is empty.\n" << C_RST;
}
pause();
}

else if (portalChoice == 3) {
    portalRunning = false;
    clearScreen();
    cout << "\n";
    divTop();
    centerRow("Thanks for using Kharcha Manager!", 33, C_GRN);
    centerRow("Goodbye! :)", 11, C_DIM + C_GRN);
    divBot();
    cout << "\n";
}
else {
    cout << C_RED << "\n ! Invalid option. Enter 1-3.\n" << C_RST;
    pause();
}
}

saveData(profiles, timeline);
return 0;

```

```
}
```

## Data Files

### members.csv

```
05|safiyah|12345|10000
A01|Ali|password123|1000
01|basit|123|2500.5
02|haseeb|123|750.5
03|aizaz|123|7500
```

### transactions.csv

```
7
1|Final Refund (Clearance)|-10000|FUND|1|01|-10000
2|electricity|3500|FUND|2|01|-1750|02|-1750
3|Final Refund (Clearance)|-93250|FUND|1|01|-93250
4|pani ka bill|9999|FUND|2|01|-4999.5|02|-4999.5
5|nashta|10000|FUND|4|04|-2500|01|-2500|02|-2500|03|-2500
6|Debt Settlement (Deposit)|1000|04|1|04|1000
```